

Внешний Курс 2 этап

дисциплина: Архитектура компьютера

Володина Алиса Алексеевна

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
4	Выводы	25

Список иллюстраций

3.1	ответ на 1 задание 2 этапа	7
3.2	ответ на 2 задание 2 этапа	8
3.3	ответ на 3 задание 2 этапа	8
3.4	ответ на 4 задание 2 этапа	9
3.5	ответ на 5 задание 2 этапа	10
3.6	ответ на 6 задание 2 этапа	11
3.7	ответ на 7 задание 2 этапа	12
3.8	ответ на 8 задание 2 этапа	13
3.9	ответ на 9 задание 2 этапа	13
3.10	ответ на 10 задание 2 этапа	14
3.11	ответ на 11 задание 2 этапа	15
3.12	ответ на 12 задание 2 этапа	15
3.13	ответ на 13 задание 2 этапа	16
3.14	ответ на 14 задание 2 этапа	17
3.15	ответ на 15 задание 2 этапа	17
3.16	ответ на 16 задание 2 этапа	18
3.17	ответ на 17 задание 2 этапа	19
3.18	ответ на 18 задание 2 этапа	20
3.19	ответ на 19 задание 2 этапа	20
3.20	ответ на 20 задание 2 этапа	21
3.21	ответ на 21 задание 2 этапа	22
3.22	ответ на 22 задание 2 этапа	22
3.23	ответ на 23 задание 2 этапа	23
3.24	ответ на 24 задание 2 этапа	24

Список таблиц

1 Цель работы

Выполнить 2 этап внешнего курса, ознакомиться с видеороликами, прикрепленными на сайте, и выполнить задания по ним.

2 Задание

Выполнить 2 этап внешнего курса в stepic.

3 Выполнение лабораторной работы

1) Для каких задач можно использовать удаленный сервер? (рис. 3.1).

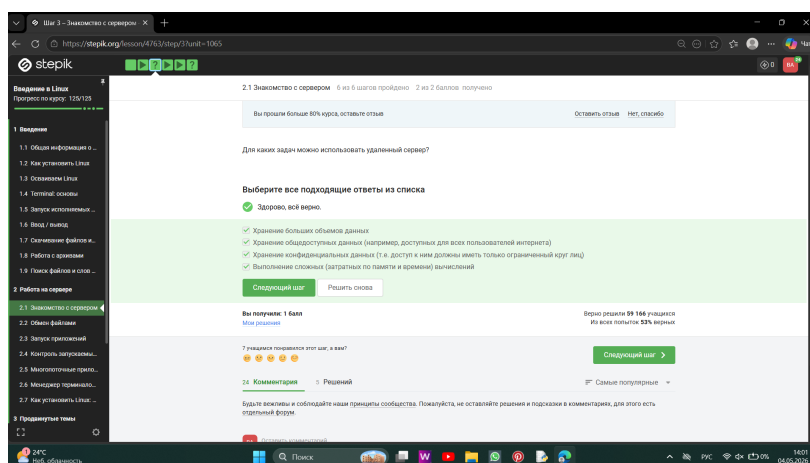


Рисунок 3.1: ответ на 1 задание 2 этапа

1. Хранение больших объемов данных — на сервере много места, которое можно расширять
2. Хранение общедоступных данных — сервер можно настроить для открытого доступа через интернет.
3. Хранение конфиденциальных данных — можно ограничить доступ паролями, ключами, VPN
4. Выполнение сложных вычислений — сервер обладает мощным процессором, памятью и работает без остановок.

2)Предположим программа ssh-keygen создала вам два ключа: id_rsa и id_rsa.pub. Какой из этих ключей можно без опаски пересылать по интернету? (рис. 3.2).

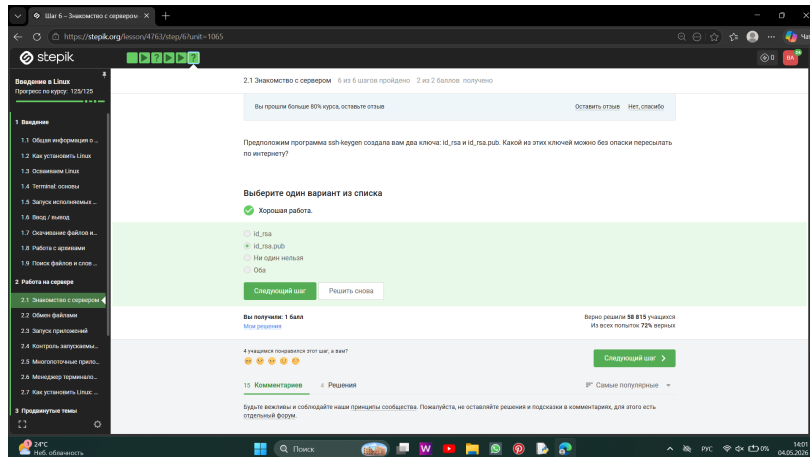


Рисунок 3.2: ответ на 2 задание 2 этапа

id_rsa.pub- Это открытый ключ. Он специально предназначен для распространения - его можно класть на серверы, передавать коллегам, публиковать. Зная открытый ключ, невозможно восстановить закрытый или взломать соединение. id_rsa - закрытый ключ, его нельзя никому передавать и хранить нужно в секрете.

3)Какая команда скопирует на сервер (в домашнюю директорию) папку stepic вместе с содержимым ее самой и всех ее подпапок?(рис. 3.3).

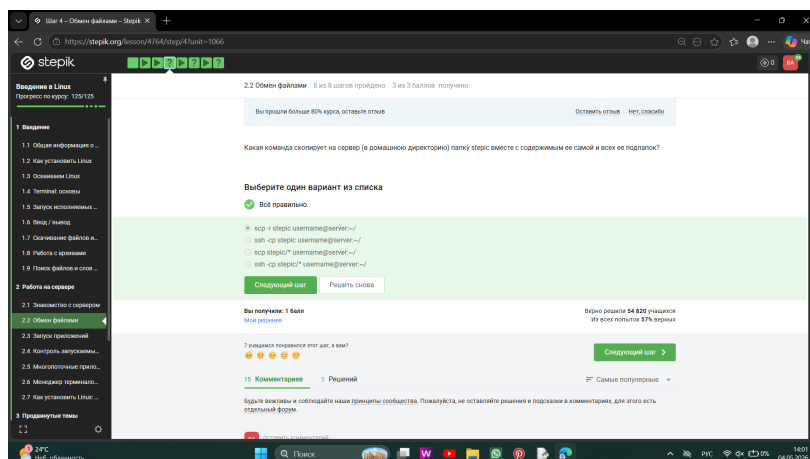


Рисунок 3.3: ответ на 3 задание 2 этапа

· `scp` — команда для копирования по SSH (в отличие от `ssh`) · `-r` (recursive) — копирует папку со всем содержимым и подпапками · `stepic` — имя папки · `username@server:~/` — удаленный сервер и домашняя директория. Остальные варианты неверны, так как используют `ssh` вместо `scp` либо копируют только содержимое (`stepic/*`), а не саму папку.

4) Предположим, что вы устанавливаете программу `program` на свой компьютер при помощи команды `sudo apt-get install program`. Терминал сообщает вам, что он не может найти и скачать установочный пакет. Какие действия могут устранить проблему? (рис. 3.4).

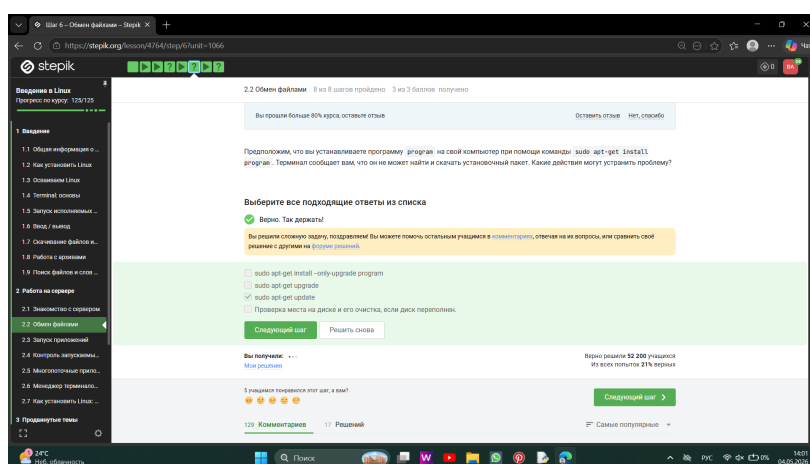


Рисунок 3.4: ответ на 4 задание 2 этапа

`sudo apt-get update` `sudo apt-get update`: Эта команда обновляет локальный список доступных пакетов из репозитория. Если список устарел, система просто не знает о существовании нужной версии программы — после `update` она сможет найти и скачать пакет

5) Для чего можно использовать программу Filezilla? (рис. 3.5).

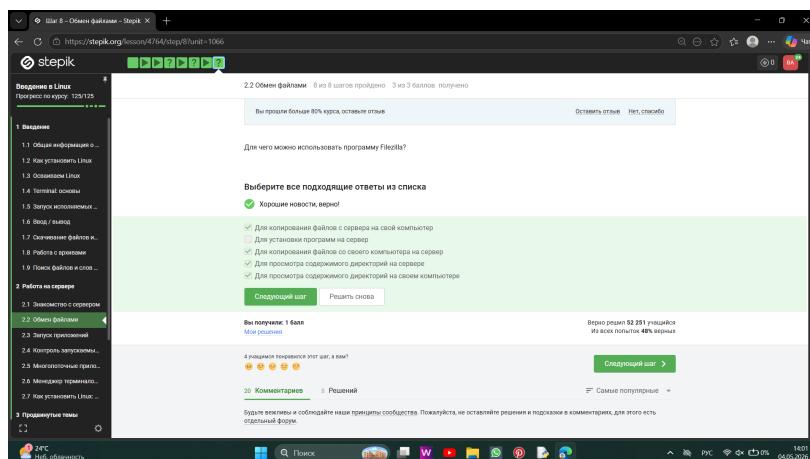


Рисунок 3.5: ответ на 5 задание 2 этапа

Filezilla — это FTP/SFTP-клиент для работы с файлами по сети. Она позволяет:

1. Копировать файлы с сервера на компьютер (скачивать)
2. Копировать файлы с компьютера на сервер (загружать)
3. Просматривать содержимое директорий на сервере (в удаленной панели)
4. Просматривать содержимое директорий на своем компьютере (в локальной панели) Не подходит «Для установки программ на сервер», потому что установка программ требует выполнения команд (например, apt, yum) или запуска инсталляторов через терминал. Filezilla только передает файлы, но не запускает их установку на сервере.

б) Что можно сделать, если требуется запустить на сервере программу, для работы которой нужен не терминал, а экран? (рис. 3.6).

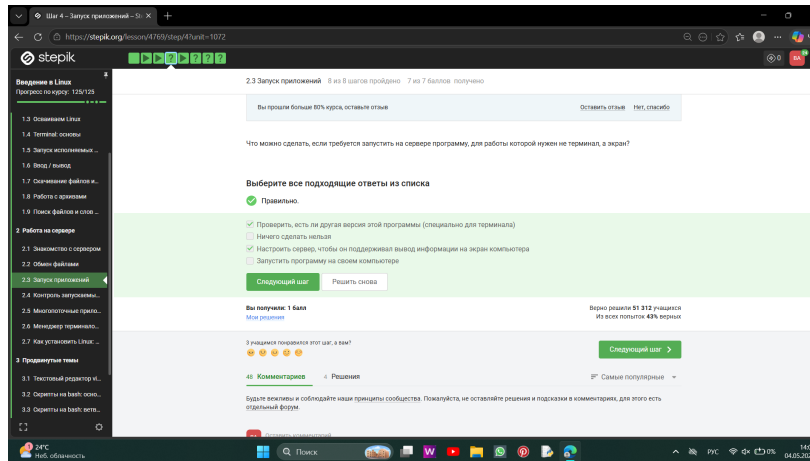


Рисунок 3.6: ответ на 6 задание 2 этапа

1. «Проверить, есть ли другая версия программы (специально для терминала)»- Многие программы имеют CLI-версию (command-line interface), которая делает то же самое, но работает без графического экрана. Это самое простое решение.
2. «Настроить сервер, чтобы он поддерживал вывод информации на экран компьютера»- Это возможно через X11 Forwarding (проброс графики по SSH). Сервер отправляет картинку интерфейса на ваш компьютер, и вы видите окно программы локально, хотя программа выполняется на сервере.

7) Как обычно можно вызвать справочную информацию о программе program? (рис. 3.7).

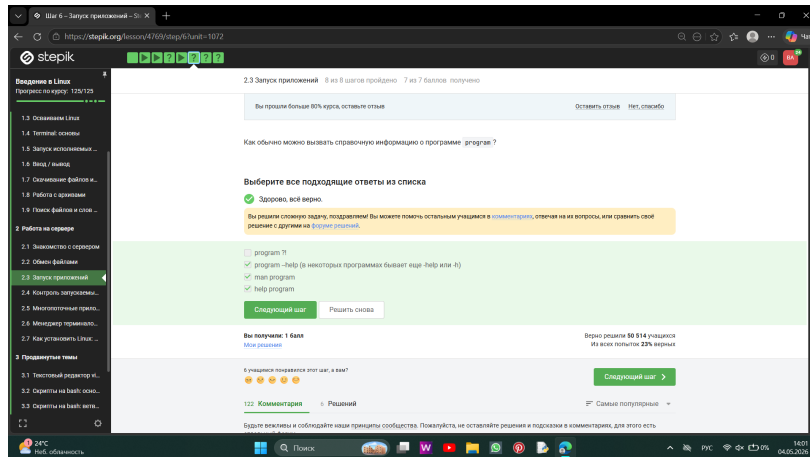


Рисунок 3.7: ответ на 7 задание 2 этапа

1. `program -help` (или `-h`, `-help`) — стандартный короткий флаг, который большинство программ поддерживают для вывода краткой справки по использованию.
2. `man program` — универсальная команда Linux, открывающая полноценное руководство (manual page) с детальным описанием всех опций.
3. `help program` — встроенная команда оболочки (shell) для получения справки о встроенных командах (например, `help cd`). `program ?!` — такого стандартного соглашения в Linux/Unix нет, этот вариант не работает.

8)Посмотрите справку по программе FastQC (имеется ввиду вариант для запуска в терминале) и определите, какие форматы данных он может принимать на вход. (рис. 3.8).

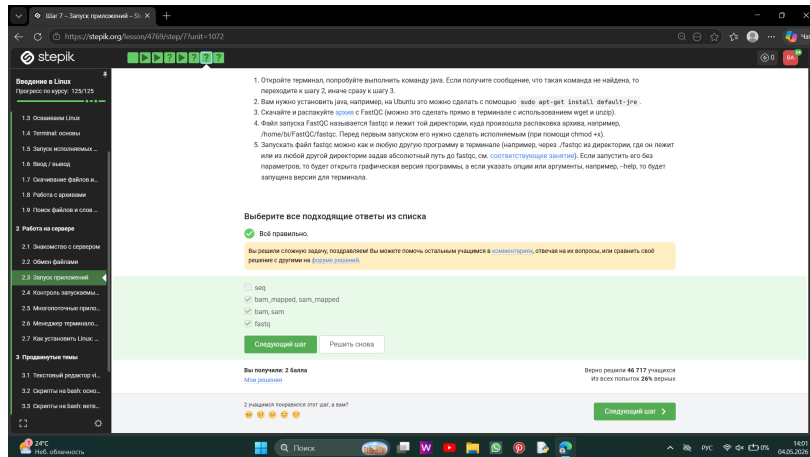


Рисунок 3.8: ответ на 8 задание 2 этапа

Это указано в справке программы (`-help`). Инструмент анализирует как сырые риды (`fastq`), так и выравненные последовательности (`bam/sam`), включая только успешно выровненные риды (`bam_mapped/sam_mapped`).

9)Посмотрите справку по программе (имеется в виду версия для терминала) и впишите в поле ниже команду, которая запускает в терминале Clustal на файле `test.fasta` и выполняет множественное выравнивание (`multiple alignment`). (рис. 3.9).

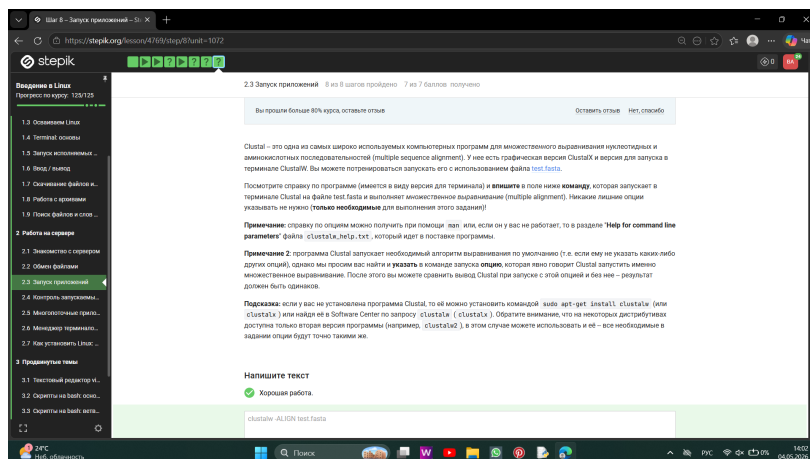


Рисунок 3.9: ответ на 9 задание 2 этапа

Я посмотрела справку по программе, ответом является `clustalw -ALIGN test.fasta`

10)Предположим вы запустили программы `program1`, `program2` и `program3` в фоновом режиме. После этого вы выполнили следующие действия: `fg %1`, `Ctrl+C`, `fg`

%2, Ctrl+Z, jobs. Информация о каких программах будет показана при выполнении команды jobs? (рис. 3.10).

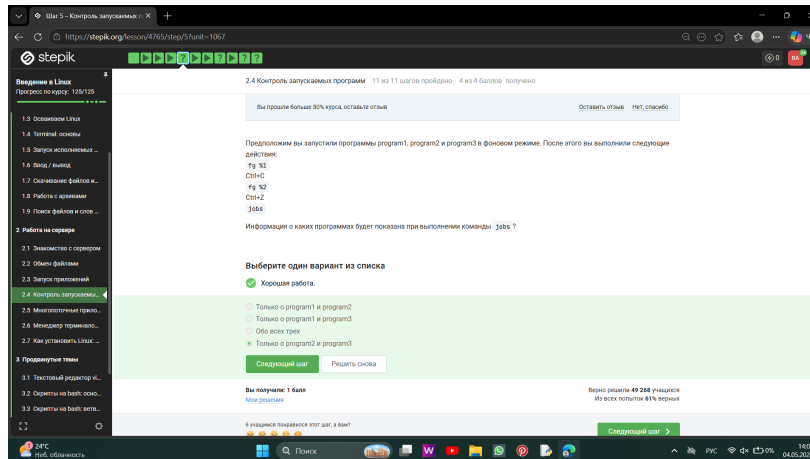


Рисунок 3.10: ответ на 10 задание 2 этапа

1. `fg %1` — вернул `program1` на передний план
2. `Ctrl+C` — завершил `program1` (она перестала существовать)
3. `fg %2` — вернул `program2` на передний план
4. `Ctrl+Z` — приостановил `program2` (осталась в фоне, но остановлена)
5. `jobs` — показывает только текущие фоновые задачи

11) `jobs`, `top` и `ps` позволяют отслеживать работу запущенных в терминале программ. В каждой из этих трех утилит для каждой запущенной программы указывается число-идентификатор. Одинаковые ли эти идентификаторы в `jobs`, `top` и `ps`? (рис. 3.11).

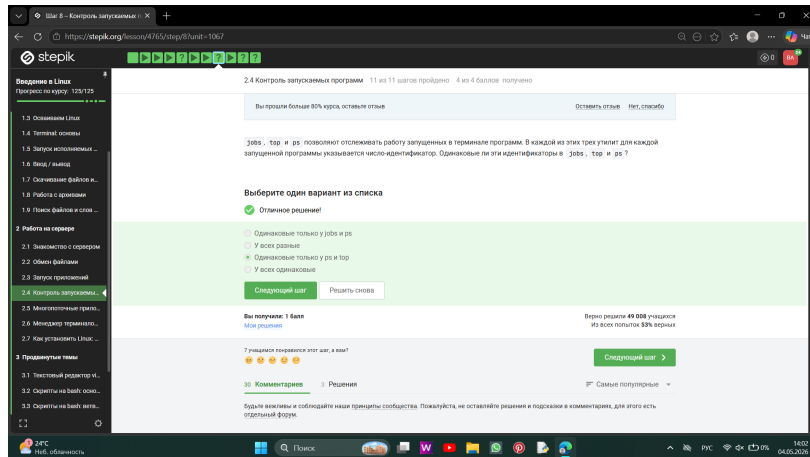


Рисунок 3.11: ответ на 11 задание 2 этапа

ps и top показывают PID (Process ID) — системный идентификатор процесса. Он одинаков для одной и той же программы в обеих утилитах. jobs показывает номер задачи (job ID: 1, 2, 3...) — это внутренний идентификатор текущей оболочки (shell), который не совпадает с PID. Поэтому у jobs идентификаторы свои, а у ps и top — одинаковые (системные PID).

12) С помощью какой команды можно мгновенно завершить остановленный процесс? (рис. 3.12).

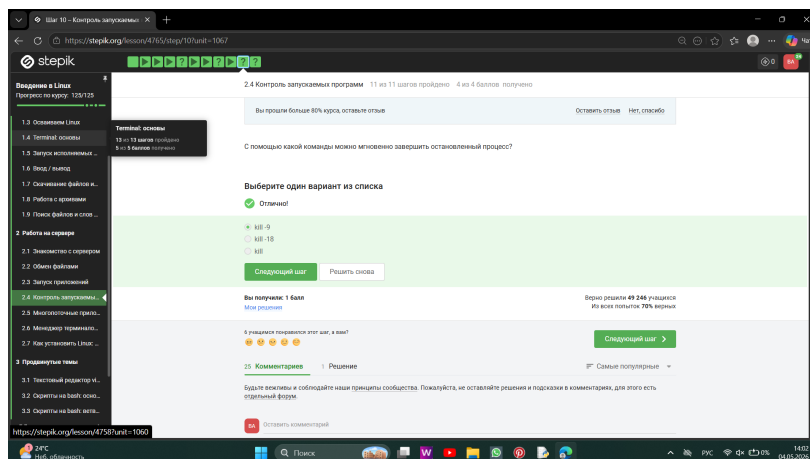


Рисунок 3.12: ответ на 12 задание 2 этапа

kill -9 (SIGKILL) — мгновенно завершает процесс без возможности его перехватить, игнорировать или обработать. Остановленный (приостановленный) процесс

завершается сразу. `kill` (без номера) — отправляет SIGTERM (15), процесс может игнорировать этот сигнал, что не гарантирует мгновенного завершения. `kill -18` (SIGCONT) — наоборот, возобновляет остановленный процесс, а не завершает его.

13) Что произойдет, если использовать `kill` (без опций) по отношению к процессу, который был приостановлен при помощи Ctrl+Z? (рис. 3.13).

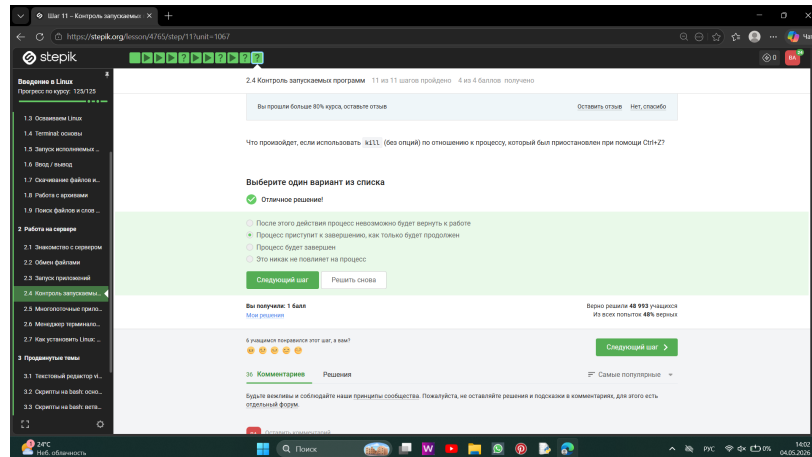


Рисунок 3.13: ответ на 13 задание 2 этапа

`kill` (без опций, т.е. SIGTERM) отправляет сигнал завершения, но процесс приостановлен (Ctrl+Z) и не принимает сигналы, пока не будет возобновлен. Когда процесс позже продолжат (например, `fg` или `kill -18`), он получит накопленный SIGTERM и начнет завершаться.

14) Сколько вычислительных ресурсов центрального процессора (% CPU) использует остановленное (по Ctrl+Z) многопоточное приложение? (рис. 3.14).

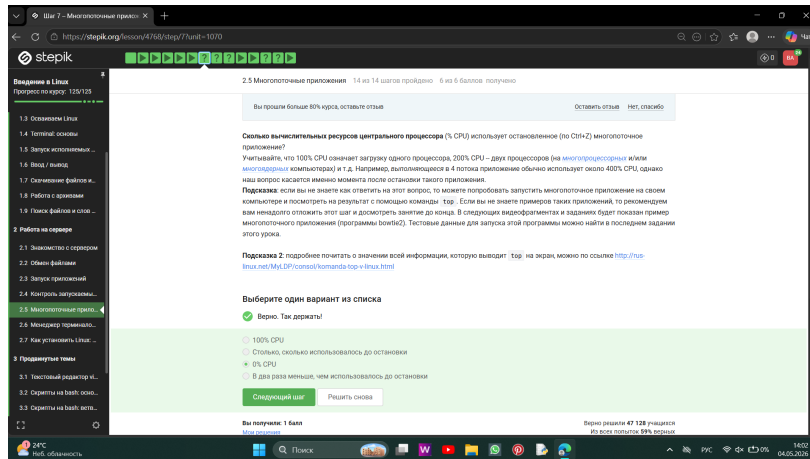


Рисунок 3.14: ответ на 14 задание 2 этапа

Остановленный процесс (SIGTSTP от Ctrl+Z) полностью исключается из планировщика задач — он не выполняет никаких инструкций и не потребляет процессорное время, поэтому его загрузка CPU равна 0%, независимо от того, сколько потоков у него было до остановки.

15) Сколько памяти занимает остановленное (по Ctrl+Z) многопоточное приложение? (рис. 3.15).

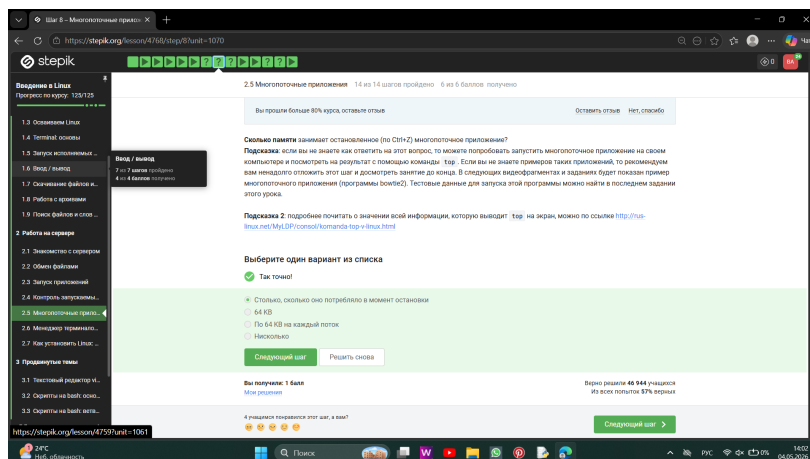


Рисунок 3.15: ответ на 15 задание 2 этапа

Остановка (Ctrl+Z) лишь приостанавливает выполнение инструкций процессором, но не освобождает выделенную процессу память (данные остаются в ОЗУ).

Программа продолжает занимать те же страницы памяти, что и до остановки.

16) Как принудительно завершить один из потоков запущенного многопоточно-го приложения? (рис. 3.16).

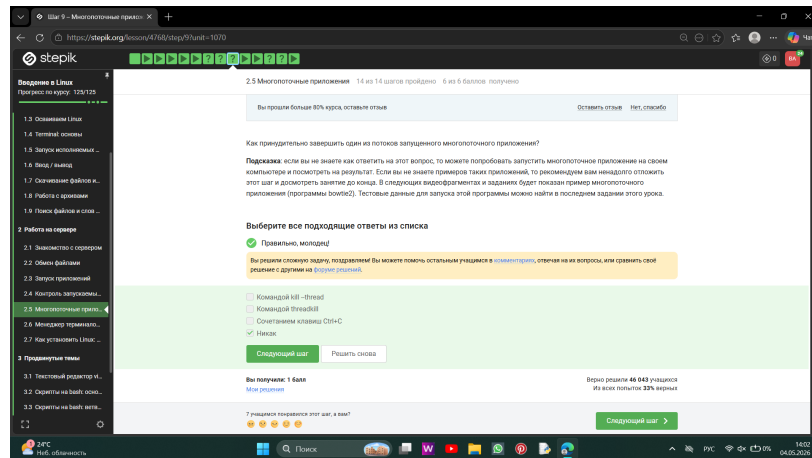


Рисунок 3.16: ответ на 16 задание 2 этапа

В стандартных Unix-подобных системах сигналы (включая kill) отправляются процессу целиком, а не отдельным потокам. Не существует штатной команды для принудительного завершения одного потока без остановки всего процесса. Потоки внутри процесса управляются ядром и самим приложением — извне выборочно «убить» один поток невозможно.

17) Для выполнения этого задания вам потребуется программа bowtie2. Изучите справочную информацию об этих подпрограммах (можно вызвать при помощи – help) и ответьте на вопрос – какой(ие) из этих шагов можно выполнить в несколько потоков? (рис. 3.17).

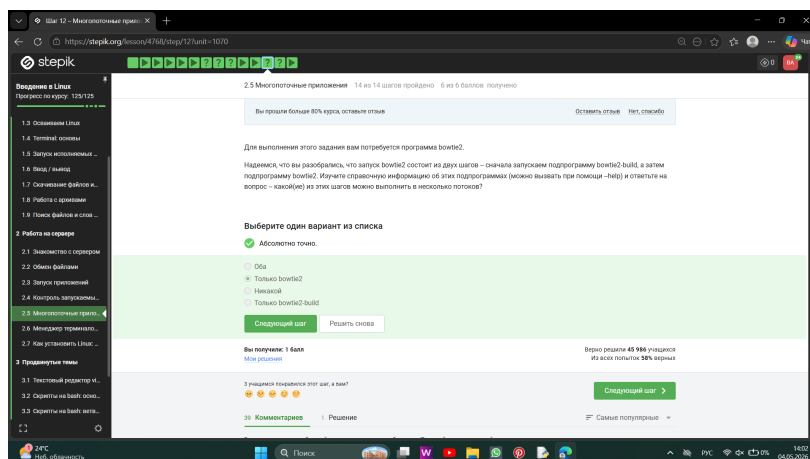


Рисунок 3.17: ответ на 17 задание 2 этапа

Согласно справочной информации, оба этапа поддерживают многопоточность, но ключевое различие в том, что только bowtie2 реально получает прирост производительности. · bowtie2-build (-threads) — поддерживает многопоточность, но прирост от использования более 1 потока очень скромный · bowtie2 (-p/-threads) — отлично масштабируется, использование нескольких потоков значительно ускоряет выравнивание (основную работу) На практике большинство документаций рекомендуют первый шаг (индексацию) запускать в 1 поток , а второй (выравнивание) — в несколько. Поскольку вопрос предполагает практическую пользу от многопоточности, правильный ответ — только bowtie2.

18) Попробуйте теперь запустить второй этап (запуск подпрограммы bowtie2) в несколько потоков. Рекомендуем выставить число потоков равное количеству ядер на вашем компьютере (команда nproc). Сравните скорость выполнения в таком режиме с работой в один поток. Также рекомендуем убедиться, что результаты запусков (т.е. вывод в stderr) полностью совпали в обоих режимах! (рис. 3.18).

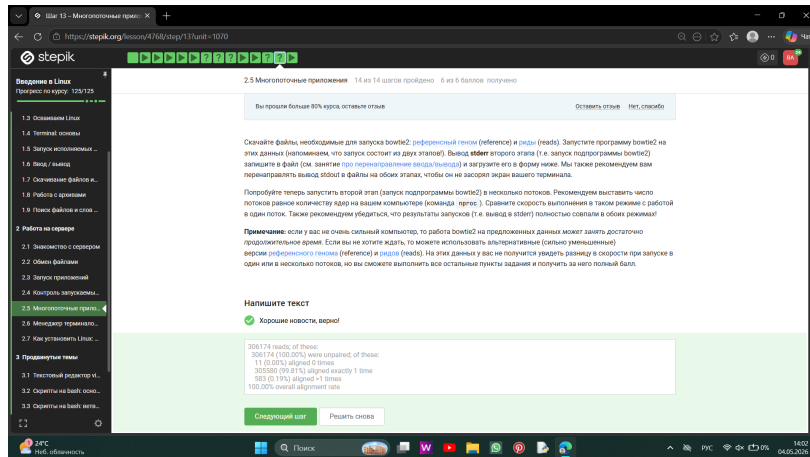


Рисунок 3.18: ответ на 18 задание 2 этапа

Попробовала запустить второй этап (запуск подпрограммы bowtie2) в несколько потоков, ответ: 306174 reads; of these: 306174 (100.00%) were unpaired; of these: 11 (0.00%) aligned 0 times 305580 (99.81%) aligned exactly 1 time 583 (0.19%) aligned >1 times 100.00% overall alignment rate

19)Вы открыли две вкладки в терминале. В одной из них вы запустили процесс и приостановили его. Переключившись во вторую вкладку и набрав fg, вы добьетесь следующего: (рис. 3.19).

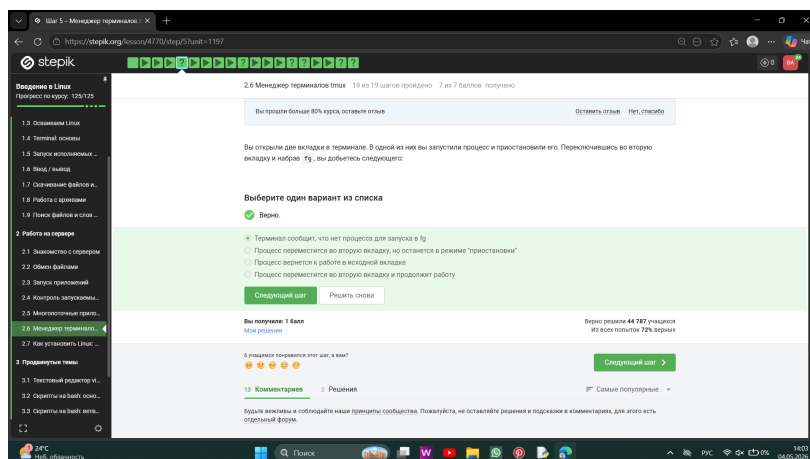


Рисунок 3.19: ответ на 19 задание 2 этапа

fg работает только в рамках текущей сессии оболочки (shell), в которой процесс

был запущен и приостановлен. При переключении на другую вкладку вы оказываетесь в другом экземпляре оболочки, у которого своя таблица задач (jobs). Там нет никаких приостановленных процессов, поэтому `fg` не находит, что возвращать на передний план.

20) Предположим, что в `tmux` осталась последняя открытая вкладка. Что произойдет, если вы введете в этой вкладке в командную строку команду `exit`? (рис. 3.20).

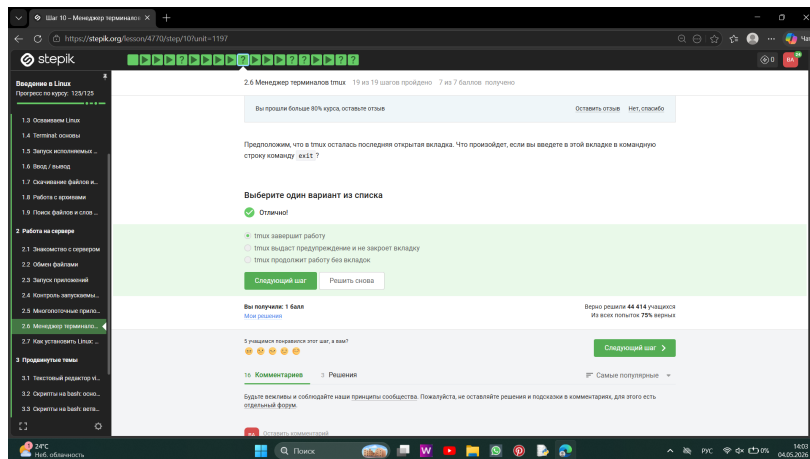


Рисунок 3.20: ответ на 20 задание 2 этапа

В `tmux` сессия существует, пока в ней есть хотя бы одно окно (вкладка). Когда вы вводите `exit` в последнем окне сессии, это окно закрывается, сессия остается без окон и автоматически завершается. Исключение — если сессия была отсоединена (`detached`), но в данном случае вы работаете в ней.

21) Предположим, что вы открыли терминал, зашли в нем на сервер, запустили на этом сервере `tmux` и начали работу в нем. Что произойдет, если вы теперь закроете терминал? (рис. 3.21).

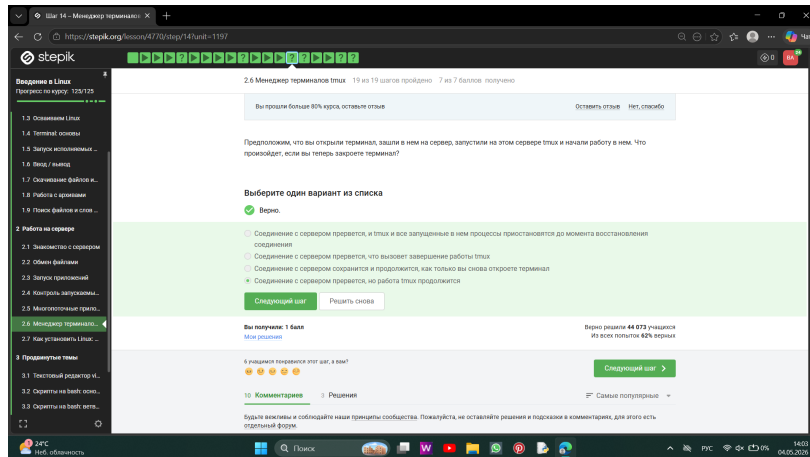


Рисунок 3.21: ответ на 21 задание 2 этапа

tmux работает на сервере независимо от вашего локального терминала. Когда вы закрываете терминал, SSH-соединение обрывается, но процессы на сервере (включая tmux и всё, что в нём запущено) продолжают выполняться. Позже вы можете снова подключиться к серверу и выполнить `tmux attach`, чтобы вернуться к той же сессии.

22) Что произойдет, если запустить процесс в фоновом режиме в одной из вкладок tmux, а затем принудительно закрыть эту вкладку (`Ctrl+B, X`)? (рис. 3.22).

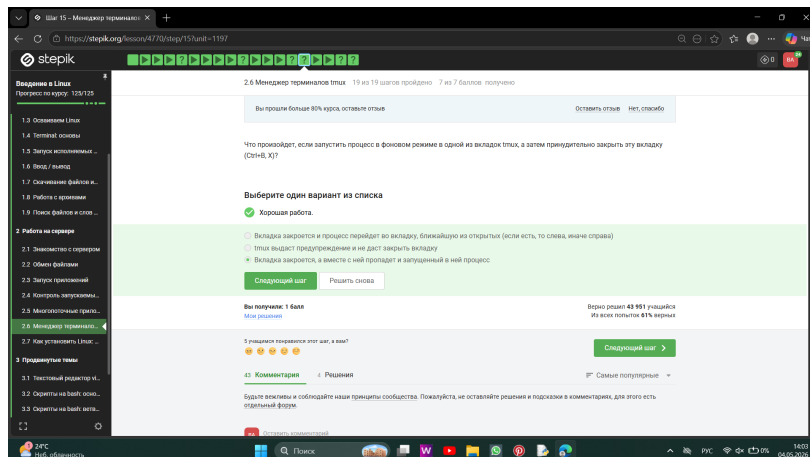


Рисунок 3.22: ответ на 22 задание 2 этапа

Когда в tmux закрывается окно (вкладка), все процессы, запущенные внутри него,

получают сигнал `SIGHUP` и завершаются. Фоновый режим (`&`) не защищает от закрытия окна — процесс все равно привязан к сессии внутри этой вкладки. Если нужно сохранить процесс после закрытия окна, его следует запускать через `nohup`, `disown` или отдельной сессией `tmux`.

23) Задание на самостоятельное изучение `tmux`. Изучите справку по `tmux` (например, `man tmux`) и выберите из предложенных ниже `tmux`-команд ту, которая отвечает за переименование текущей вкладки. (рис. 3.23).

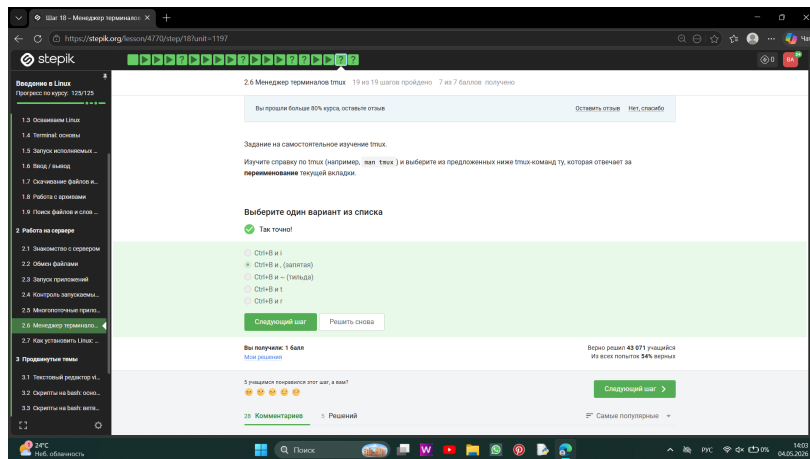


Рисунок 3.23: ответ на 23 задание 2 этапа

В `tmux` сочетание `Ctrl+B`, (`Control + B`, затем запятая) вызывает команду переименования текущего окна (вкладки). После этого внизу появится строка ввода, где можно изменить название вкладки. Остальные варианты делают другое: `Ctrl+B i` — информация о текущем окне `Ctrl+B ~` — сообщения об ошибках/отладка `Ctrl+B t` — показать часы `Ctrl+B r` — перезагрузить конфигурацию

24) Предлагаем вам самостоятельно изучить работу с «вкладками внутри вкладок» и отметить верные утверждения из списка ниже. Вы можете использовать справку по `tmux` (например, `man tmux`) или просто попробовать воспроизвести эти утверждения у себя на компьютере (рис. 3.24).

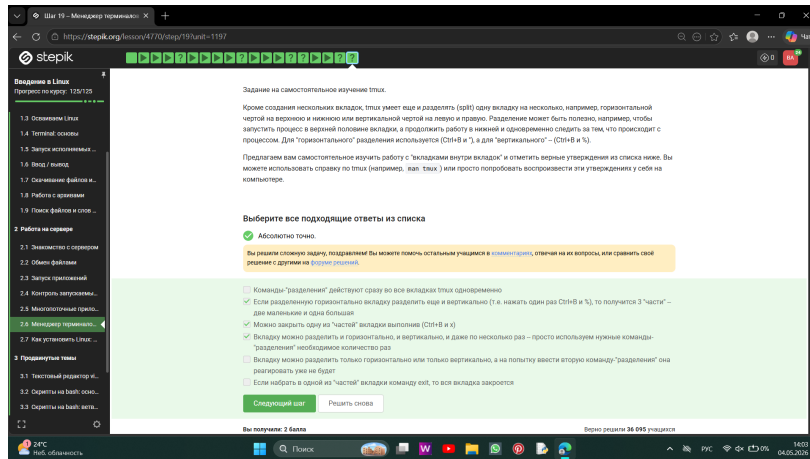


Рисунок 3.24: ответ на 24 задание 2 этапа

1. 3 части — при горизонтальном делении (верх/низ) и последующем вертикальном делении одной из половин (например, верх) получается левая, правая и целая нижняя панели.
2. Ctrl+B + x — закрывает только текущую панель (часть окна), остальные остаются.
3. Можно делить многократно — tmux позволяет дробить любую панель в произвольном порядке сколько угодно раз, создавая сложные раскладки.

4 Выводы

Я выполнила 2 этап внешнего курса, ознакомилась с видеороликами на сайте, и выполнила задания по ним